International Journal of Gaming and Computer-Mediated Simulations
Volume 10 • Issue 3 • July-September 2018

On Computerizing the Ancient Game of Ṭāb

Ahmad B. Hassanat, Mutah University, Karak, Jordan
Ghada Altarawneh, Mutah University, Karak, Jordan
Ahmad S. Tarawneh, Eotvos Lorand University ELTE, Budapest, Hungary
Hossam Faris, The University of Jordan, Amman, Jordan
Mahmoud B. Alhasanat, Al-Hussein Bin Talal University, Maan, Jordan
Alex de Voogt, Drew University, Madison, USA
Baker Al-Rawashdeh, Mutah University, Mutah, Jordan
Mohammed Alshamaileh, Mutah University, Mutah, Jordan
Surya V. B. Prasath, Cincinnati Children's Hospital Medical Center, Cincinnati, USA

ABSTRACT

The ancient game of ṭāb is a war and race game. It is played by two teams, each consisting of at least one player. In addition to presenting the game and its rules, the authors develop three versions of the game: human versus human, human versus computer, and computer versus computer. The authors employ a Genetic Algorithm (GA) to help the computer to choose the 'best' move to play. The computer game is designed allowing two degrees of difficulty: Beginners and Advanced. The results of several experiments show the strategic properties of this game, the strength of the proposed method by making the computer play the game intelligently, and the potential of generalizing their approach to other similar games.

KEYWORDS

AI, Ancient Game, Computer Games, Genetic Algorithm, Intelligent Agents, Social Game

INTRODUCTION

The game of ṭāb (Arabic: طاب) is a board game, played by two teams, each of which consists of at least one player. It uses a game board of four rows of holes, which are normally impressed in the sand, with typically 8 to 12 holes per row. The rows of holes are used to host the players' pieces while playing. The pieces, also referred to here as soldiers, are moved based on the throws of four two-sided stick dice. The game ends by capturing all soldiers of the opponent and in this game a tie is not possible. Figure 1 shows two teams of two players enjoying the game of ṭāb in Petra, Jordan.

This game is one of the most popular board games in the Middle East and attested particularly in Jordan, Palestine, Sudan and some places in Egypt. The history of this game can be traced back to several hundred years. A recent survey of the archaeological region of Petra in Jordan revealed an unusually large number of ṭāb playing boards carved in rock surfaces distributed over two major sites

DOI: 10.4018/IJGCMS.2018070102

Copyright © 2018, IGI Global. Copying or distributing in print or electronic forms without written permission of IGI Global is prohibited.

Figure 1. Two teams of two players playing the game of ṭāb in Petra. Four stick dice are used and the board is impressed in the sand. Photograph: Alex de Voogt 2009.

in the ancient city (De Voogt, Hassanat, & Alhasanat, 2017). See Figure 2 for examples of these game boards. The survey study suggests a connection to the ancient city of Petra, but there was no evidence to date this game back to the Nabataeans (about 2000 years ago). The game has not been mentioned in Roman sources and no excavation has revealed such game boards even though several such game boards were found near archaeological sites. In addition, there is little evidence that can date the origin of the game of ṭāb to any specific period so that the birth date of the game of ṭāb remains elusive.

A nineteenth century description of this game is almost identical to what it is found today (Murray, 1952). Murray reports on sources that show that this game was played in Turkey, Egypt, and Persia, so that its presence in Jordan is not necessarily surprising. The majority of the ṭāb players in Jordan seem to be elderly people. It is hard to find a young man in the area who knows how to play ṭāb, or who even knows what the game of ṭāb is. Therefore, at least in Jordan, the gaming practice is slowly disappearing in the absence of a new generation of players.

The main goal of this work is to develop an intelligent computerized version of the game of ṭāb that allows the game to be acquired and enjoyed by a younger generation in Jordan and beyond. In

Figure 2. Examples of ṭāb game boards attested in the archaeological region of Petra in Jordan

addition, the unique properties of the game as described in section 2 may serve the research community for allowing further comparisons and analysis. This may provide the necessary momentum to ensure its cultural survival as well.

The literature is rich with attempts to computerize board games such as Sega (Abdelbar, Ragab, & Mitri, 2003), Chess (Simon & Schaeffer, 1992), Go (Jagiello, et al., 2006), Backgammon (Tesauro, 2002), etc. In addition, there are attempts to program proprietary games, such as Human Pacman (Cheok, et al., 2004) and several computing games (Björk, Holopainen, Ljungstrand, & Åkesson, 2002) and other ancient Egyptian games (Crist, Dunn-Vaturi, & de Voogt, 2016). The current research introduces the first computerized version of the game of ṭāb, a game that has properties not found in the games analysed in previous studies.

We have developed three versions of the game: Human against Human, Human against Computer, and Computer against Computer. The heuristic developed for this game chooses the "best" or "near optimal" move, and is designed with two levels of competency, Beginners and Advanced.

The remainder of this paper is organized as follows: Section 2 provides the basic descriptions, components and rules of the game of ṭāb; Section 3 details the computerization of the game using a genetic algorithm approach; Section 4 provides some experimental results using the developed computerized program; and, Section 5 summarizes the main conclusions that can be drawn from this work.

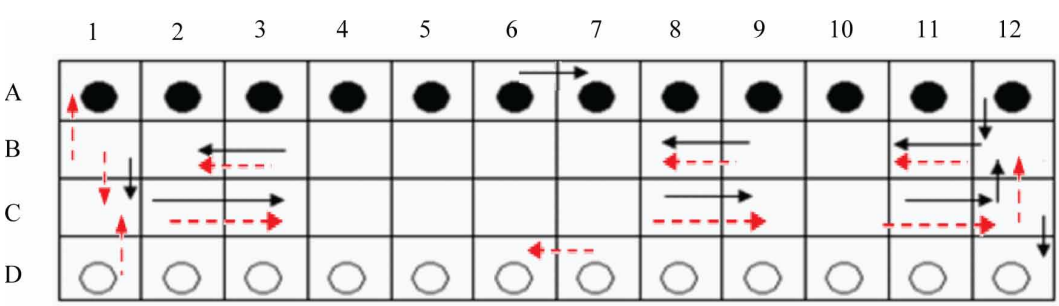
GAME COMPONENTS AND RULES

The game has three main components: a board, pieces and four stick dice. First, the game of ṭāb uses a game board that consists of a fixed number of rows (4) but a varying number of holes per row, referred to here as columns. The first row is home to the soldiers of the first team, and the fourth row to the opposing team. The in-between rows represent the battlefield. The soldiers move in a specified direction along the board. Figure 3 depicts the game board and the move directions.

Second, the game features playing pieces commonly referred to soldiers or puppies by local players. They are represented by white and black circles in Figure 3. Players commonly use stones of two distinct colours, preferably black and white. The number of soldiers for each team is identical to the number of columns on the board.

The red-dotted arrows shown in Figure 3 depict the moves of the white soldiers, while the black arrows show the moves of the black soldiers. Each player starts with one soldier on their left-hand side —D1 for white, and A12 for black. Soldiers cannot return to their home row, however, they can enter the opponent's home row. When they do so, they remain frozen until all the soldiers in the other rows have been captured. Some players prefer to protect some of their soldiers from capture this way, since the opponent's soldiers cannot reach this row once they are in play.

Figure 3. A sketch of the ṭāb game board representing the starting position of the game. The numbers and letters were added to give an address to each location in order to encode the moves.

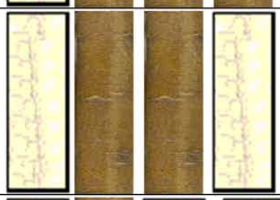
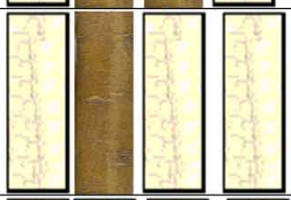
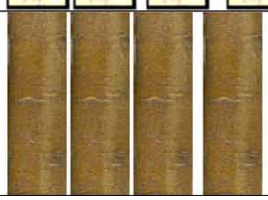


Third, four stick dice are used to get a value with which a player can move their soldiers. These sticks are rectangular in shape and identical in size. They are commonly taken from Oleander trees, then cut horizontally to get a half cylinder shape with one flat side, represented here with a white colour, and one convex side, shown here as a dark green or brown colour. An overview of the values and the probabilities of each throw is presented in Table 1.

Stick dice are used instead of cubic dice. The latter are common in previously programmed games such as backgammon (Tesauro, 2002), Yahtzee (Glenn, 2006) and Monopoly (Yasumura, Oguchi, & Nitta, 2001). All six numbers on a cubic die have an equal probability. When using stick dice, players get five different numbers (1, 2, 3, 4, or 6), but each number has a different probability as shown in Table 1.

When a player throws the sticks, the number of flat or white sides determine the moves the player may play. If just one of the four sticks show its flat side, the player may move one piece one square further along the board. This score of one is called ṭāb. Ṭāb is considered the best move in the game; first, it is not allowed to jump over opponent's soldiers, so that this move allows for capturing soldiers directly in front of one's own piece; second, it allows the player to play again, i.e., the player throws

Table 1. The sticks' shape, value and the probability for each possible throw

Sticks side	Value	Probability to Occur
	1 (ṭāb)	25%
	2	37.5%
	3	25%
	4	6.25%
	6	6.25%

the sticks again after completing this first move of one square; finally, it is the only throw that allows a soldier to move from its home row. The game and the throw are named after each other.

If any two sticks show their flat sides, the player gets to move two squares. If three sticks show their flat sides, the player gets to move three and four when four sticks show their flat sides. The player gets six when no flat side is shown. There is no value of "five" that can be thrown with these sticks.

The probabilities of getting 1, 4 and 6 are relatively small and if thrown the player is allowed to throw the sticks again just as with the throw of ṭāb. In contrast, the player stops playing and gives the sticks to the opponent after throwing 2 or 3.

At the beginning of the game, each player throws the sticks once; the one who gets the largest number starts playing the game. Each player needs to throw a ṭāb to free their first soldier; if not, the other player takes their turn to play. After a soldier is freed by one ṭāb move, it can be moved with all other throws as well. Players may agree to relax this rule in order to speed up the game. The players move their soldiers from the home row in a specific order, from the rightmost piece to the leftmost piece.

As indicated above, one turn may consist of multiple throws of the sticks. A player may choose to play a soldier directly or the player can wait until all the throws in one turn are completed. In the latter case, the player is allowed to move one or more of the pieces using their throws in any sequence they wish.

If a player's piece lands on any hole occupied by an opponent's piece, the opponent's piece is captured and removed from the game. A player may move one or more of their own soldiers onto the same square, creating a set. The set then moves as a whole and is potentially captured as a whole. To disassemble a set, a ṭāb throw is needed to remove one piece to the next position. A set is not allowed to move around in the back row of the opponent. Once it reaches this row or is created there, it stays put. Only if no other moves are possible can a set on an opponent's home row be moved.

After a soldier reaches the right end of the home row, it proceeds up one hole and begins moving clockwise in the two center rows. Here the soldier will circle in the center two rows until it is captured or moved into the opponent's home row. In other words, the game of ṭāb has a race and a war game component, which is known as a running-fight game. The options for moving pieces that a dice throw affords suggest that the game should be considered a competitive strategic game rather than a gambling game. In addition, the player is allowed to wait until all the throws have finished, after which the optimal order of moves is determined. This adds further strategy to the game.

Two of the co-authors video recorded and discussed (in Arabic) a complete ṭāb game to illustrate the game rules, see https://youtu.be/wIF86O-EgMw. For alternates description of the rules see (Murray, 1952) and (De Voogt, Hassanat, & Alhasanat, 2017).

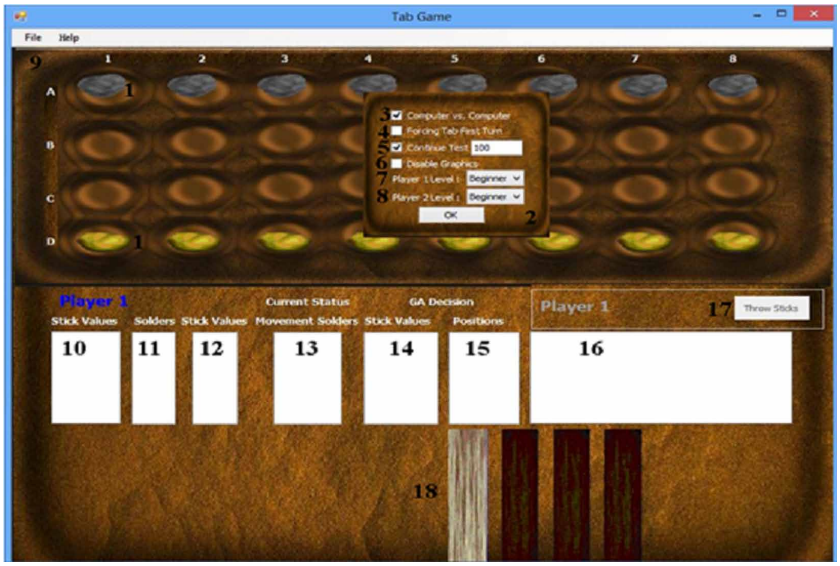
The Ṭāb Game Application

We used VC++.NET 2008 to develop the game application. We selected a board of four rows with eight holes per row. The main screen developed for this application shows boxes that provide information about the current state, stick values, history of moves, etc. Figure 4 shows the main screen of this ṭāb application.

As shown in Figure 4, there are 18 elements numbered from 1 to 18 that represent the following options:

1. This element represents a soldier. Each player has eight soldiers; instead of black and white, we used silver and gold for the two sides;
2. This shows an options menu that contains settings used in the game;
3. This option allows two computer agents to play against each other;
4. Checking this option forces soldiers to make their first move using a ṭāb throw;
5. This option regulates the number of games that the computer agents will play against each other (for research purposes);

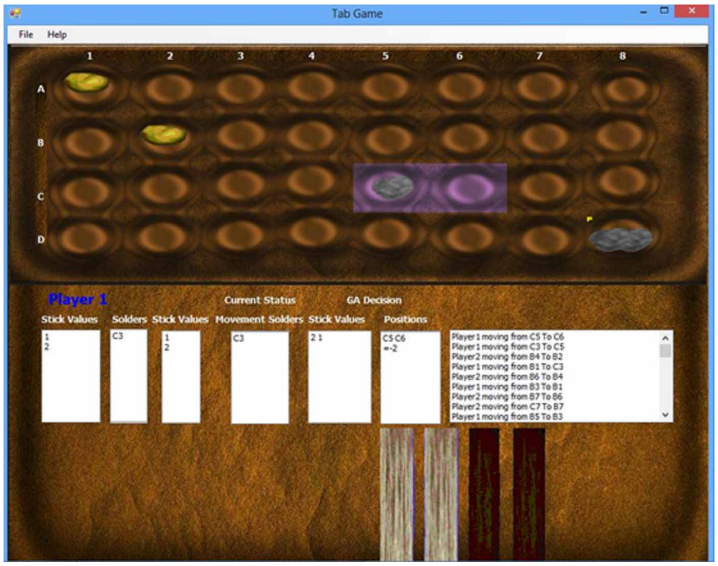
Figure 4. The main screen developed for the ṭāb game



6. This option disables the visualization of the moves to save time, particularly when two computer agents play many games for research purposes;
7. This option sets the level of the first player if it is a computer agent, which can be a beginner or advanced;
8. This option sets the level of the second player if it is a computer agent, which can be beginner or advanced;
9. This option shows the board coordinates, which help the players to identify the positions of their soldiers;
10. This option provides the value list of the stick dice. It also shows all the values that the player obtained during the current turn, so that the player can choose which one to use first with which soldier;
11. This option lists the soldiers that a player can move, each soldier is represented by its initial index (from 1 to 8 for player 1, and 25-32 for player 2);
12. This option also shows the history of stick dice throws but here the system has removed the ṭāb throws that moved the frozen soldiers;
13. This option ensures that a soldier is automatically freed after playing ṭāb;
14. This option lists the stick dice throws that have automatically freed a soldier;
15. This option shows the position of each soldier after automatically freeing the soldiers with a ṭāb throw. Also, this list shows the cost of the decision that will be used by GA, which will be discussed later;
16. This option shows the history of each move for each player showing the coordinates of old and new positions of soldiers;
17. This option is used to throw the sticks and get new values; this option is available for human players only;
18. This figure provides a graphic view of the stick dice that shows the sides of the sticks for each throw.

Figure 5 shows a snapshot of the ṭāb game application, in this snapshot some of the soldiers have moved from initial positions to the active positions on the board.

Figure 5. A snapshot while the game is played between two computer agents



As shown in Figure 5 there is a group of soldiers on D8; grouped soldiers can only be separated by throwing ṭāb, otherwise they remain as a set; soldiers in D8 cannot move because they are in the opponent's area, also A1 cannot move until B2 is captured or gets into row 1. The snapshot shows how the silver soldier moves from C5 to C6. Algorithm 1 shows how the soldiers are moved over the board.

Algorithm 1: The Moves of Soldiers on the Board

```
Start
throw_again:
Value = Throw_the_sticks()
IF value ==1
IF frozen soldiers then
Move frozen soldier 1 move
End if
End if
Sticks_values_array.add(value)
If Value == 1 OR Value == 4 OR Value == 6
Go to throw_again;
End if

Selected_soldier_ position = get_soldier_position(mous_click)//
reject
Oldnumberofsoldiers= Selected_soldier_ position.
getsoldiersnumber();
Selected_move_ position = get_position(mous_click)
Distance = |Selected_move_ position - Selected_soldier_ position|
Value_index = Sticks_values_array.find (Distance)
New_ position = Selected_soldier_ position+ Distance
Move_soldier_to(New_ position)
Sticks_values_array.remove_at(Value_index)
```

```
Snumber = Number_of_soldiers(New_ position)
S_team= type_of_soldiers(New_ position)
If S_team==our_team
Snumber= Snumber+ Oldnumberofsoldiers
New_ position_soldiers.add(Snumber, New_ position)
Else
position_soldier(Oldnumberofsoldiers, New_position)//kill any
opponent soldiers in a new positions
End if
Team1_soldiers=team1.numb_of_soldiers()
Team2_soldiers= team2.numb_of_soldiers()
If Team1_soldiers==0
winner= Team2
Else if Team2_soldiers==0
Winner= Team1
Else
Change_player();
Goto throw_again;
End if

End
```

A Game of Probabilities

Winning a ṭāb game is decided by choosing the "best" soldiers to move and by selecting which value to use for each soldier at each turn. This is a nontrivial task even for the most expert players. The difficulty stems from the number of possible solutions (moves) that may be available to the players, especially when the player gets a larger sequence of values of 1s, 4s and 6s (the throws that allow the player to throw again) in combination with a large number of free soldiers. Thus, the number of available solutions in ṭāb increases dramatically when the throw sequence increases and when the number of free soldiers increases.

For example, suppose that there are M free soldiers and N throw values. Any soldier can be moved by any number in the sequence, or one soldier or sub-group of soldiers can be moved by all the values. In addition, the player can start with any value in the sequence, so the sequence is not sorted. The number of these solutions can be defined by:

$$\text{Number of solutions} = \frac{\left(N+M-1\right)!}{\left(M-1\right)!} \tag{1}$$

Table 2 illustrates how an increasing number of free soldiers with specific values for the stick dice creates an increasing number of possible solutions for a player.

As shown in Table 2 there are a large number of solutions even with small sequences. When there are 5 or more free soldiers, which is statistically common; we get particularly large numbers of different solutions that complicate decision-making. Due to time constraints, a brute-force search algorithm is not practical to find the optimal move. Instead, we opt for the genetic algorithm (GA) as a meta-heuristic search tool to find a near-optimal, but possible, move. GA is not the only method that can solve this problem; other optimization methods can too, these include particle swarm optimization (Kennedy & Eberhart, 1995), cuckoo search (Yang & Deb, 2009), Ant colony optimization (Dorigo & Birattari, 2011), etc. However, GA is a well-established optimization method and has been used

Table 2. The possible number of solutions that can be generated from a different number of free soldiers (M) and stick dice values (N)

M	N									
	1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
1	1	2	6	24	120	720	5040	40320	362880	3628800
2	2	6	24	120	720	5040	40320	362880	3628800	4E+07
3	3	12	60	360	2520	20160	181440	1814400	2E+07	2.4E+08
4	4	20	120	840	6720	60480	604800	6652800	8E+07	1E+09
5	5	30	210	1680	15120	151200	1663200	2E+07	2.6E+08	3.6E+09
6	6	42	336	3024	30240	332640	3991680	5.2E+07	7.3E+08	1.1E+10
7	7	56	504	5040	55440	665280	8648640	1.2E+08	1.8E+09	2.9E+10
8	8	72	720	7920	95040	1235520	1.7E+07	2.6E+08	4.2E+09	7.1E+10

extensively in the computer science literature for the optimization of game play, such as Tetris (Böhm, Kókai, & Mandl, 2005), Seega (Abdelbar, Ragab, & Mitri, 2003), Checkers (Chellapilla & Fogel, 1999), (Chellapilla & Fogel, 2001), War Game Strategies (Revello & McCartney, 2002), Othello (Sun, Liao, Lu, & Zheng, 1994), first-person shooter game (Cole, Louis, & Miles, 2004), etc. Since this presents the first computerization of the ṭāb game, a comparison of these other optimization methods is outside the scope of this study.

Genetic Algorithm

GA is an evolutionary computing algorithm that is based on evolution and natural selection theory (Holland, 1992). It is an efficient tool for solving optimization problems. Particularly, it helps with combinatorial problems that cannot be solved in polynomial time, such as the one that we have with ṭāb (Hassanat A. B., et al., 2016).

Typically, GA consists of four major components: Initial Population, Crossover, Mutation and Selection based on fitness function (Hassanat & Alkafaween, 2017).

Initial Population

The initial population seeding is the first phase of any GA application. It generates, randomly or systematically, a population of feasible solutions or individuals (Hassanat, Prasath, Abbadi, Abu-Qdari, & Faris, 2018). Here, we create the initial population by allocating the stick values that move free soldiers randomly. This generates different random solutions where each soldier may or may not move.

Suppose that a player throws the sticks and gets the following sequence of values (moves): 6, 6, 4, 4, 4 and 2, and having 3 free soldiers. One possible solution can be created by allocating (6 and 4) to move soldier (1), (6, 4 and 4) to move soldier (2), and, finally, (2), to move soldier (3) two steps ahead, or any other combination from 20160 different solutions, since we have M=3 and N=6, see Table 2. Table 3 shows some of these possible solutions.

Crossover

Typically, the crossover is a binary process that operates on two solutions (chromosomes) called parents in order to generate new offspring solutions. This process attempts to create better solutions from existing in-hand solutions (Hassanat, Prasath, Abbadi, Abu-Qdari, & Faris, 2018). Here, we opt for the traditional one-point crossover process. In this process, it is essential to choose a random separation point that allows the parents' chromosomes to form new offspring, or solutions containing genes from both parents. Figure 6 shows the new offspring that result from the crossover process.

Table 3. Some of the possible random solutions to move 3 free soldiers ahead when throwing a sequence of 6, 6, 4, 4, 4 and 2 with the sticks dice

Soldier	Moves				
	Solution 1	Solution 2	Solution 3	Solution 4	Solution 20160
1	6,6	6,4,4,4	4,6	6,6,4,4,4,2	2
2	4,4	6	6,4,4	-	-
3	4,2	2	2	-	6,6,4,4,4

Fixing Chromosome Size and Content

As can be seen from Figure 6 there are some missing values in offspring (2), and more but also incorrect values in offspring (1). These errors occur because of the nature of one-point crossover used. Therefore, we need to correct each new offspring to get a legal solution. To solve this problem, we create a histogram for the stick values with the length of the histogram having five values: 0, 1, 2, 3 and 4. These values represent one of the possible values of the stick dice (1, 2, 3, 4 or 6) to which we add the frequencies of the sticks values. The histogram of the previous example (shown in Figure 6) is shown in Table 4.

New offspring can be determined based on this histogram. Each offspring gets a copy of this histogram; we scan each solution and decrease the frequency of the found value. If we find a value that has a 0 frequency, this means it is an illegal move, so we can delete it from the chromosome. If the chromosome scanning has finished and there are still frequencies in the histogram, we add them randomly with random soldier(s) in the chromosome. Figure 7 shows the correction of offspring (1) from Figure 6.

As shown in Figure 7, all extra values are removed, but the final histogram shows that there is still a missing value, which is 6. We add this number to a random soldier. If we randomly choose soldier (3), the final solution and histogram is given in Figure 8.

Figure 6. Crossover process of Solutions (1 and 2 from Table 3)

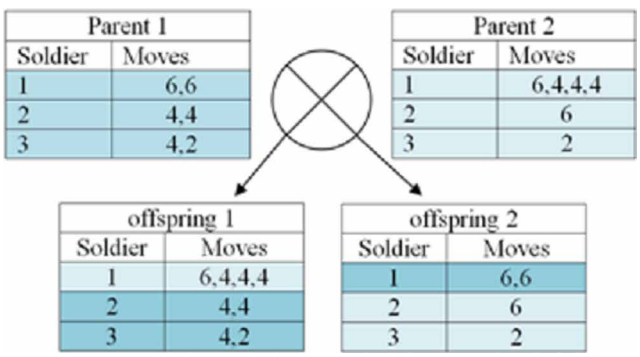


Table 4. The histogram of sticks values of the example shown in Figure 6

Values	1	2	3	4	6
Frequencies	0	1	0	3	2

Figure 7. Fixing chromosome's size and content offspring (1) from Figure 6, with stick values {6; 6; 4; 4; 4; 2}, and histogram {0; 1; 0; 3; 2}

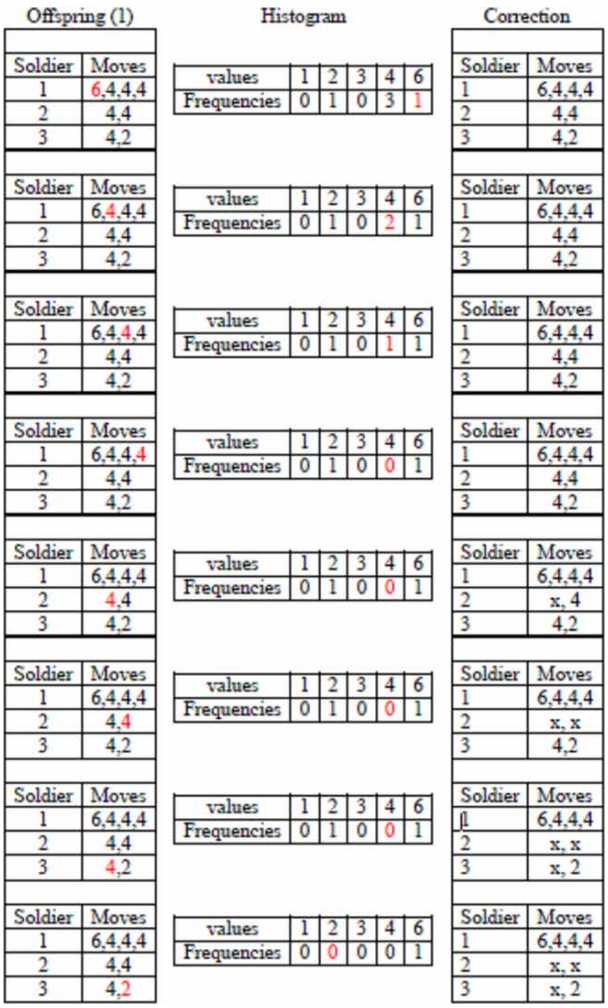


Figure 8. Final solution and histogram

Legal Offspring (1)	
Soldier	Moves
1	6,4,4,4
2	-
3	2,6

values	1	2	3	4	6
Frequencies	0	0	0	0	0

All new offspring are checked in the same way to guarantee legal solutions, which are used later for moving the soldiers on the board. If offspring (1) is used in the game, then the agent must move soldier (1) {6; 4; 4; and 4} steps ahead, and move soldier (3) {2 and 6} steps ahead, capturing any opponent's soldier located in any occupied square on which it lands.

Mutation

Mutation is a unary process that aims at increasing diversity by producing a new solution from another solution. It changes some information inside the targeted solution randomly (Algethami & Landa-Silva, 2017). Here we used a 10% mutation rate for each generation, and each mutation can follow one of the following processes:

1. Swapping moves of two soldiers within the same solution;
2. Swapping two moves of the same soldier within the same solution;
3. Assign one move or more of one soldier to another.

Using any of the previous mutations guarantees a legal solution, and therefore, there is no need for fixing the mutated solutions.

Fitness Function

Fitness function is used with GA to evaluate solutions. It uses calculations that depend on the problem in hand. In our application, the fitness function depends on finding a weight for each solution. Each square on the board is given a value that is affected by one or more soldiers occupying that square. The board contains dangerous squares, which expert players try to avoid occupying. These squares are the ones located in the way of the opponent's soldiers. There are also safe or safer squares, namely, those which are located farther from the reach of the opponent's soldiers. Both dangerous and safe squares are given high weights (points); however, the points of the dangerous ones have negative values. The remainder of the squares have neutral values. Both dangerous and safe squares are identified in Figure 10.

There is no completely safe place on the board with the exception of the home row of the opponent. But as soon as the soldiers get there, they cannot move unless there are no other moves left. There are other factors affecting the weight of each solution, and they can be summarized as follows:

1. For each soldier staying in the home row 4 points are added to the solution. For each soldier moving to the playing area or the middle rows, 2 points are added;
2. For each soldier moving to the strategic area (from B1-B4) 2 points are added;
3. The presence of more than one soldier in the same square deducts 4 times the number of the merged soldiers from the total number of points. The deduction process is maintained by adding negative points to the total weight of a solution;
4. If there are enemy soldiers located 4 squares or less behind the square proposed by the solution, then 2 points per opponent's soldier in such a position are deducted from the total;
5. If the proposed square is in the opponent's zone (row C), which is part of the dangerous squares, and the number of opponent's soldiers is small, i.e., less than 4, then 3 times the number of the opponent's soldiers is deducted from the total;
6. Putting soldiers in the opponent's home row (from D8 down to D1) gives additional points; the largest value goes to square D8, i.e., 8 points, and gradually decreases to 1 point for square D1;
7. If the number of the agent's soldiers is less than 4, then 3 points are deducted from the total weight if the soldier entered the opponent's zone (D), because the soldier becomes frozen and cannot play and capture.

Choosing the previous points (parameters) was based on pilot experiments. They were conducted and programmed by expert players that include the first author and three of the co-authors. The pilot experiments were based on trial and error, as the parameters were chosen, and the game was then tested by four of the authors who are already expert ṭāb players. The GA is used to generate solutions

with the highest number of points (weight). The following is an example of a real status of the game showing the calculations of the fitness function.

Assuming the current state is as shown in Figure 11, and that it is the turn of the computer (the silver stones) to play, and it threw (4), (4), (1) and (2) with the stick dice, then the GA gives the following solutions:

Solution 1: This solution proposes moving the silver soldier from A8 to B2 using (1), (2), and (4), in addition to moving the other soldier from C1 to C5 using (4). The fitness function is calculated as follows: 2 points are added for each soldier in the playing area. In addition to 2 points for moving to a strategic square (B2), this makes the total number of points equal to 6. But 6 points are deducted because we have 3 opponent soldiers of which none have been captured. Four more points are deducted because of the threat of these soldiers at B1 and D1 (two for each), making the final weight of this solution -4. Figure 9 shows an example for each of these mutations. Figure 10 shows dangerous and safe areas. Figure 12 shows the solution for the game state in Figure 11.

Solution 2: This solution proposes moving the silver soldier from A8 to B6 using (1) and (2), in addition to moving the other soldier from C1 to D8 using (4) and (4). In this case, 2 points are added because of moving to playing zones B and C (if it had moved inside zone A it would get 4 points). In addition, there are 2 points added from the second soldier and 8 points for moving to the safest square (D8). This makes the total weight 12. But since we have 2 opponent soldiers, 2 points are deducted for each of them. Finally, because of entering the opponent's zone (D) and having a small number of soldiers (less than 4), 3 points are deducted making the final weight equal to 5. Figure 13 shows the solution (2) for Figure 11.

Figure 9. Examples of the 3 mutation types adopted

Type 1

Legal Offspring (1)	
Soldier	Moves
1	6,4,4,4
2	-
3	2,6

New offspring	
Soldier	Moves
1	2,6
2	-
3	6,4,4,4

Type 2

Legal Offspring (1)	
Soldier	Moves
1	6,4,4,4
2	-
3	2,6

After mutation

New offspring	
Soldier	Moves
1	4,4,4,6
2	-
3	6,2

Type 3

Legal Offspring (1)	
Soldier	Moves
1	6,4,4,4
2	-
3	2,6

New offspring	
Soldier	Moves
1	6,4,4,4
2	6
3	2

Figure 10. Dangerous area (red), safe or strategic area (blue), and neutral area (uncolored)

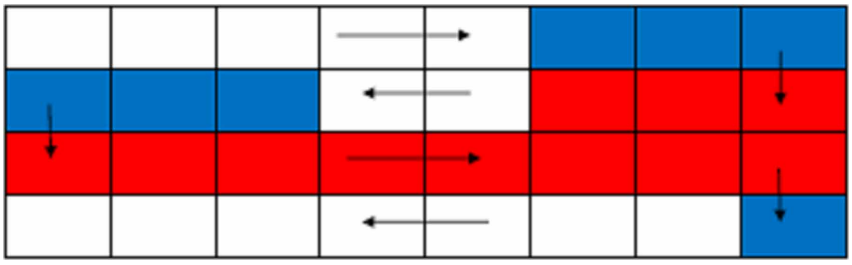


Figure 11. An assumed state of the game

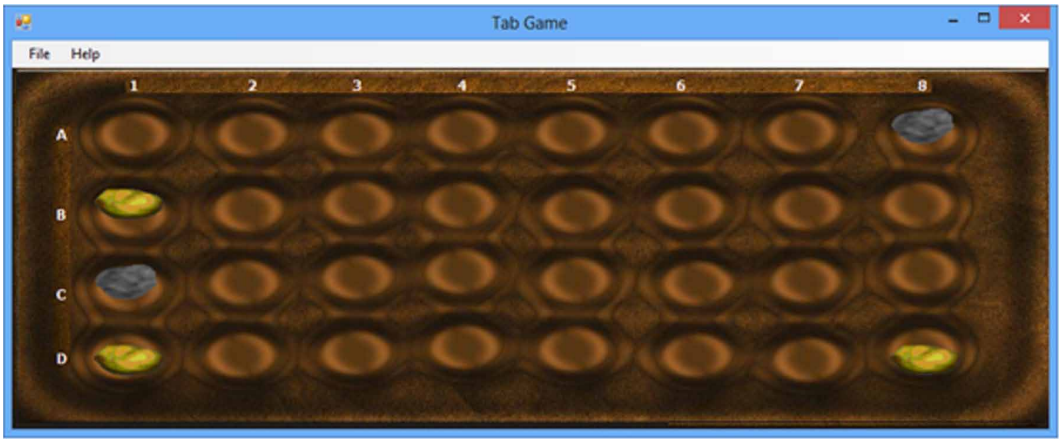


Figure 12. Solution (1) for the game state in Figure 11

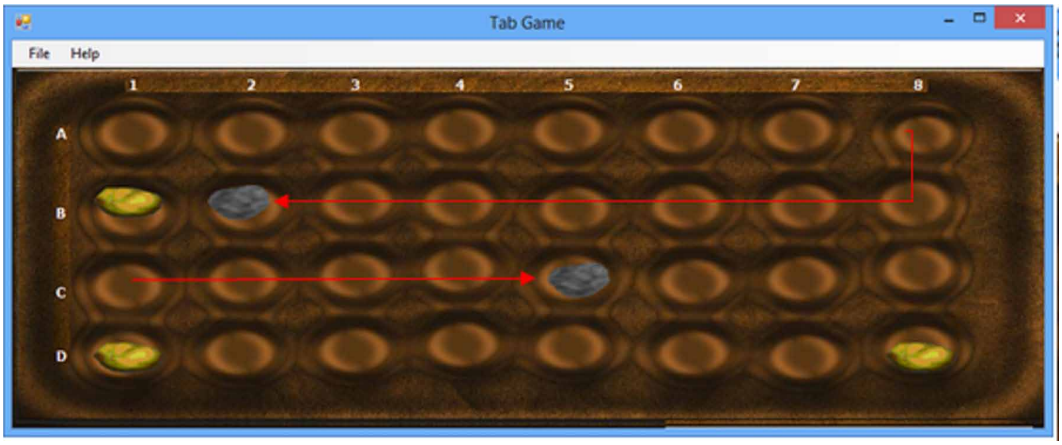
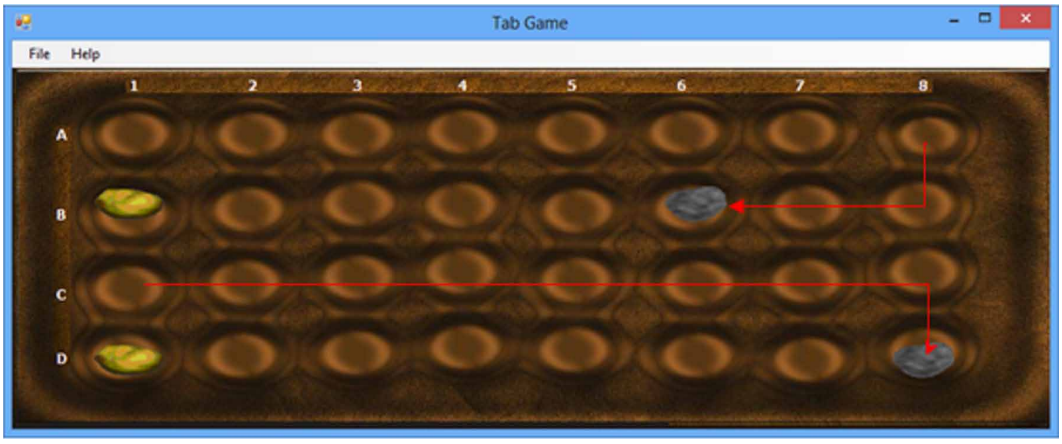


Figure 13. Solution (2) for the game state in Figure 11



Solution 3: This solution proposes moving the silver soldier from A8 to B5 using (4), in addition to moving the other soldier from C1 to C8 using (1), (2) and (4). There are 2 points added for each soldier moving to the playing zone, and 6 points are deducted because of the 3 opponent's soldiers, making the final weight equal to -2. Figure 14 shows the solution (2) for Figure 11.

Solution 4: This solution proposes moving the silver soldier from A8 to B1 using (4) and (4) to capture an opponent's soldier on B1 and carrying on moving using (2) and (1), but without moving soldier at C1. This adds 4 points for deploying 2 soldiers to the playing zone. It deducts 4 points for having an opponent's soldier at D1 threatening both of the computer's soldiers and 4 more points are deducted for having 2 opponent's soldiers. This makes the final weight equal to -4. Figure 15 shows the solution (2) for Figure 11.

Solution 5: This solution proposes moving the silver soldier from A8 to B1 using (4) and (4) to capture the opponent's soldier on B1 and carrying on moving using (1) to merge with another soldier at C1. Then both are moved to C3 using (2). There are 4 points added for deploying 2 soldiers to the playing zone, 4 points are deducted for having 2 opponent soldiers, 2 more points are deducted for having a threat coming from D1, and in addition, 8 points are deducted from merging 2 soldiers into a set (-4 points for each merged soldier). This makes the final weight equal

Figure 14. Solution (3) for the game state in Figure 11

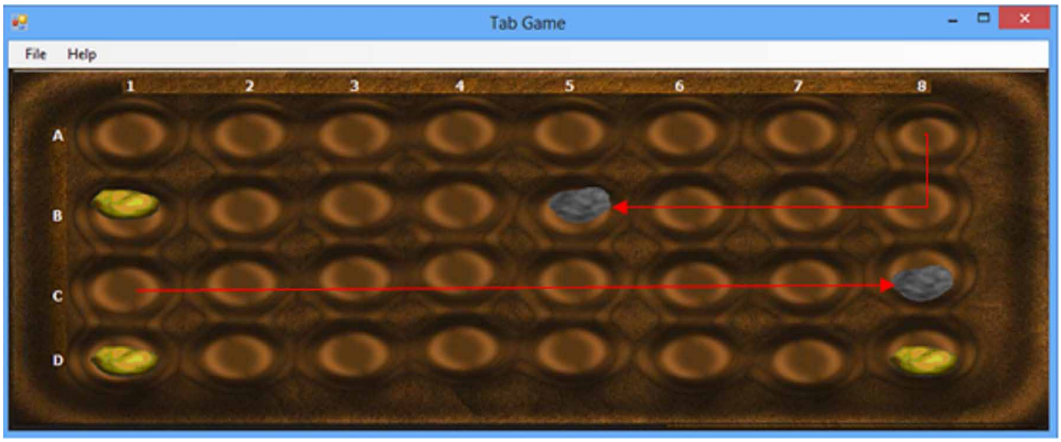
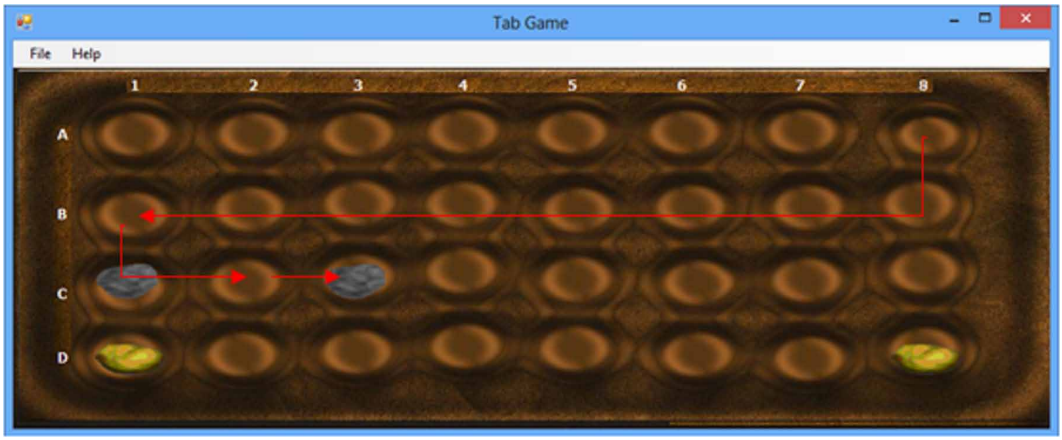


Figure 15. Solution (4) for the game state in Figure 11



to -10. This might be the worst solution as the opponent can easily end the game by throwing 3, which has a 25% probability, or by getting 1 and 2, or 6 and 4, etc. Figure 16 shows the solution (2) for Figure 11.

Solution 6: This solution proposes moving the silver soldier from A8 to B1 using (4) and (4) to capture an opponent's soldier on B1, and moving the other silver soldier from C1 to C3 using (2) and (1). This adds 4 points for deploying soldiers to a playing zone, in addition to 2 points for having one soldier on a strategic square (B1). Then 4 points are deducted for having 2 opponent soldiers, 2 more points are deducted for having a threat coming from D1. This makes the final weight equal to 0. Figure 17 shows the solution (2) for Figure 11.

As can be seen from the above weight calculations, Solution (2) is the best according to the fitness function, and this would be a logical conclusion, as the computer can now capture one soldier and secured one of its soldiers in the D zone, while it keeps one of its soldiers 5 squares away from the nearest opponent's soldier. This makes it difficult to be captured in the opponent's turn. Table 5 summarized all the probabilities for capturing the soldier on B6 from B1 in Solution (2).

Figure 16. Solution (5) for the game state in Figure 11

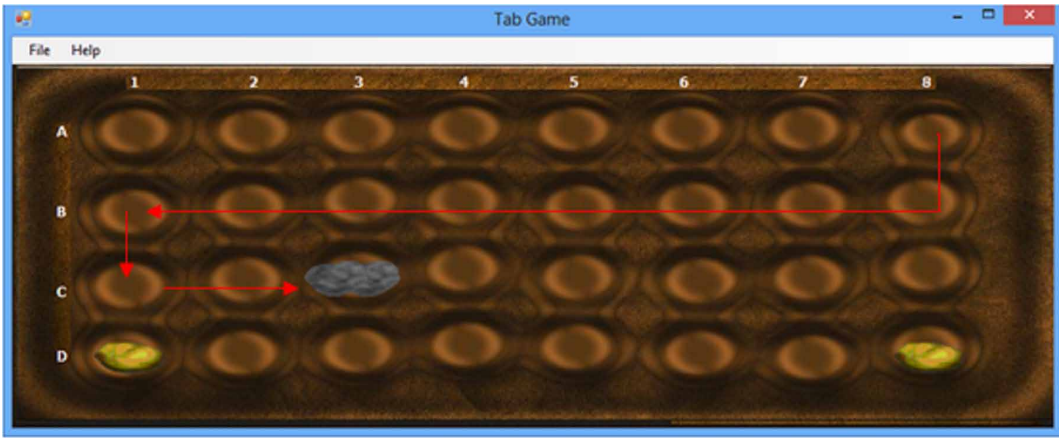
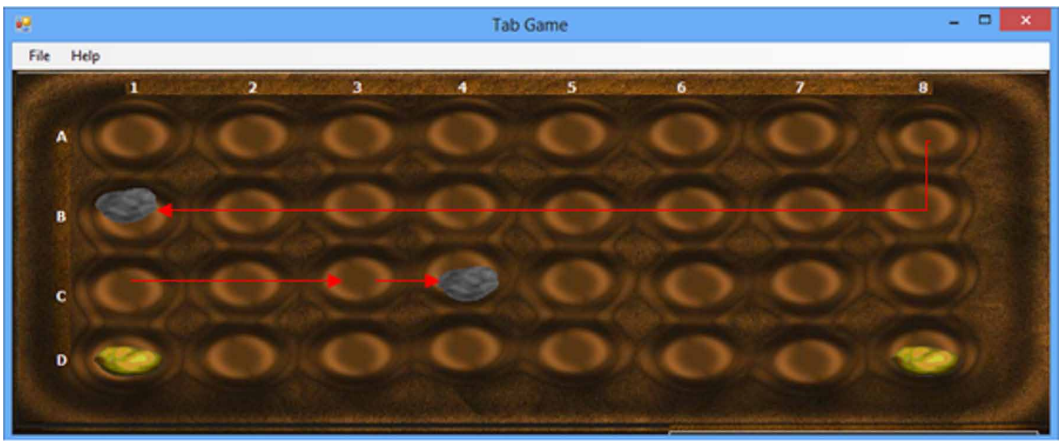


Figure 17. Solution (6) for the game state in Figure 11



As can be seen from Table 5, it is not likely (3.8%) that the computer's soldier B6 will be captured by the opponent's soldier located five squares behind. Also, it should be noted that there is no 5 in the game. The table shows the strength of solution (2) as proposed by the fitness function.

Experimental Results

In this study, we programmed the game using 2 degrees of difficulty, i.e., using 2 agents, Advanced and Beginner. The Beginner agent is almost the same as the Advanced, with the following exceptions: a) the number of generations of the GA; we used 200 generations for each solution for the Advanced agent, while we used only 50 generations for the Beginner agent, as it is well-known that more generations for GA provides better solutions (Oh, 2017), (Amirjanov & Sobolev, 2017). b) The Beginner agent does not include values for grouped soldiers. The other parameters of the GA are the same for both agents, which are:

- Crossover rate = 80%;
- Mutation rate = 10%;
- Size of population = 500 chromosomes (solution);
- The selection operation chooses the best 500 solutions at each generation.

Choosing the aforementioned GA's parameters was based on pilot experiments conducted with both Advanced and Beginner agents. These parameters were then chosen through trial and error to meet the best performance for both agents. To evaluate the performance of the proposed agents, we conducted 5 sets of experiments, which are based on playing the game as follows:

1. Advanced vs. Advanced
2. Advanced vs. Beginner
3. Beginner vs. Beginner
4. Human vs. Advanced
5. Human vs. Beginner

Each set of experiments was repeated 100 times before drawing any conclusion. The human players consist of 10 expert players, who have been experience with playing the game for at least 4 years. Their ages range from 24 to 65 years. Four of these players are the authors of this paper, while the rest are volunteers considered experts by the local community. Each played 10 times in each of experiments (4 and 5). The results of the 5 sets of experiments are listed in Table 6.

As can be seen from Table 6 the Advanced agents compared to the Beginner won most of the games, 79%. Compared to a human agent, it won 60% of the played games. These results support the strength of the proposed heuristic, particularly, when we give the GA enough time (200 generations) to find the 'best', i.e., near optimal, move. In addition, these results stress the importance of playing

Table 5. All possible situations where the opponent captures the computer's soldier on B6 from B1 in Solution (2)

Which Throw is Needed to Capture a Soldier Located 5 Squares Ahead?	Probability*	
5 ones	0.25×0.25×0.25×0.25×0.25	0.000977
3 ones and a two	0.25×0.25×0.25×0.375	0.005859
2 ones and a three	0.25×0.25×0.25	0.015625
a one and a four	0.25×0.0625	0.015625
Sum		0.038086

* Probability for each number to occur is taken from Table 1

Table 6. Experimental results

Experiments	# of Wins	Largest Sequence (Sticks Value)	Largest Number of Free (Movable) Soldiers
Player1 = Advanced Player2 = Advanced	47 53	5	5
Player1 = Advanced Player2 = Beginner	79 21	4	6
Player1 = Beginner Player2 = Beginner	43 57	5	8
Player1 = Beginner Player2 = Human	38 62	6	7
Player1 = Advanced Player2 = Human	60 40	7	7

strategically with the grouped-soldiers, knowing that the GA of the beginner agent has not enough time to evaluate solutions and ignores the grouped-soldiers strategy. Moreover, the human brain cannot compete with machines when there is a very large number of possibilities to consider (Bushko, 2005).

When human players played against a beginner agent, they won most of their games (62%). This is explained by the weakness of the beginner agent for the above-mentioned reasons. The advanced agent when playing another advanced agent, both performed almost the same (47% vs. 53%), there was no fifty-fifty result because a larger number of simulations would be needed to have a near equal distribution of advantageous dice throws. Statistically speaking the probability to get advantageous ṭāb throws is such that players should count on strategies more than luck when playing ṭāb.

For each experiment, we recorded the number of stick dice values for each throw (the number of times a player was allowed to play before handing the sticks to the opponent), in addition to the number of "Free" (or movable) Soldiers. Those two numbers (N and M from Table 2) are crucial to the number of possible solutions, as the number of these solutions is calculated by a factorial function as shown in Equation (1). A closer look at the data in Table 6, specifically the third and fourth columns, we see that one of the experiments has 5 numbers in sequence in one turn, and 8 free soldiers (maximum). This gives according to Equation (1) 2E + 07 possible solutions. Therefore, it is a rational decision to use an optimization method such as the GA to optimize such a large number of solutions.

The aforementioned experiments used three versions of the game, which were designed for a beginner agent, an advanced agent and a human. Table 7 summarizes the differences between these versions.

Table 7. Summary of different versions of the game

Game Version	#Generations	Grouped Soldiers Used	GA Parameters	Throw Sticks Button
Beginner agent	50	No	Crossover rate = 80%, Mutation rate = 10%, Size of population = 500	NA
Advanced agent	200	Yes	Crossover rate = 80%, Mutation rate = 10%, Size of population = 500	NA
Human player	NA	Yes	NA	Yes

CONCLUSION

In this work, we have presented the game of ṭāb and computerized it for the first time to be played by humans and/or machines. We used a GA approach to create the best (near optimal) solution for two agents (Advanced and Beginner). Several experiments were conducted to evaluate the performance of the agents. Due to the large number of different solutions presented to a player, the agent designed on the advanced level can beat human players in most of the played games, proving the strength of our proposed GA approach for this game. In addition, we show that for human players the game of ṭāb is a strategic game as opposed to a game of chance. This claim is further supported by the experiments conducted.

One of the limitations of this study is the lack of comparisons of our agent with agents from other studies due to the absence of previous studies for the game of ṭāb. For this reason, the first version of the game is made freely available on Google drive at (https://drive.google.com/drive/folders/0B6f_uKdnLbrlcjNxMHkwa2REM2s?usp=sharing). It allows researchers from the computer game community to contribute to better versions of this historical game. Other limitations of this work include the use of probabilities for the throws. For example, we added penalties to the moves that put a soldier in a location less than 5 squares ahead of an opponent's soldier. We did not distinguish between these 4 squares. However, statistically, one square ahead is different from 2 squares ahead, as the probability of getting 1 (25%) is less than the probability of getting 2 (37:5%). The same applies to 3 and 4 squares ahead. This issue needs to be further investigated in future work in order to enhance the performance of the heuristic used. Finally, the fitness function used in this paper depends heavily on expert knowledge. Future research may explore a fitness function that allows the algorithm to discover its own strategies, for instance, by using a form of deep learning.

We aim not only to address the limitations mentioned above in future research but also to develop mobile and web-based versions. The latter are instrumental in facilitating the spread of the game and raising awareness of the game for both general and academic audiences.

ACKNOWLEDGMENT

The authors would like to thank Jose Brox with University of Coimbra, Portugal; And Ioulia N. Baoulina with Moscow State Pedagogical University, Russia, for their help and valuable discussions to formulate Equation (1).

REFERENCES

Abdelbar, A. M., Ragab, S., & Mitri, S. (2003). Applying co-evolutionary particle swam optimization to the Egyptian board game seega. In *Proceedings of the first Asian-Pacific workshop on genetic programming* (pp. 9-15).

Algethami, H., & Landa-Silva, D. (2017). Diversity-based adaptive genetic algorithm for a Workforce Scheduling and Routing Problem. In *2017 IEEE Congress on Evolutionary Computation (CEC)* (pp. 1771-1778). IEEE.

Amirjanov, A., & Sobolev, K. (2017). Scheduling of directed acyclic graphs by a genetic algorithm with a repairing mechanism. *Concurrency and Computation*, *29*(5), 1–10. doi:10.1002/cpe.3954

Björk, S., Holopainen, J., Ljungstrand, P., & Åkesson, K. P. (2002). Designing ubiquitous computing games–a report from a workshop exploring ubiquitous computing entertainment. *Personal and Ubiquitous Computing*, *6*(5), 443–458. doi:10.1007/s007790200048

Böhm, N., Kókai, G., & Mandl, S. (2005). An evolutionary approach to Tetris. In *The Sixth Metaheuristics International Conference (MIC2005)* (pp. 5-11).

Bushko, R. (Ed.). (2005). *Future of Intelligent and Extelligent Health Environment*. Amsterdam: IOS Press.

Chellapilla, K., & Fogel, D. B. (1999). Evolving neural networks to play checkers without relying on expert knowledge. *IEEE Transactions on Neural Networks*, *10*(6), 1382–1391. doi:10.1109/72.809083 PMID:18252639

Chellapilla, K., & Fogel, D. B. (2001). Evolving an expert checkers playing program without using human expertise. *IEEE Transactions on Evolutionary Computation*, *5*(4), 422–428. doi:10.1109/4235.942536

Cheok, A. D., Goh, K. H., Liu, W., Farbiz, F., Fong, S. W., Teo, S. L., & Yang, X. et al. (2004). Human Pacman: A mobile, wide-area entertainment system based on physical, social, and ubiquitous computing. *Personal and Ubiquitous Computing*, *8*(2), 71–81. doi:10.1007/s00779-004-0267-x

Cole, N., Louis, S. J., & Miles, C. (2004). *Using a genetic algorithm to tune first-person shooter bots. In Congress on Evolutionary Computation, CEC2004. 1* (pp. 139–145). IEEE.

Crist, W., Dunn-Vaturi, A. E., & de Voogt, A. (2016). *Ancient Egyptians at Play: Board Games Across Borders*. London: Bloomsbury Publishing.

De Voogt, A., Hassanat, A. B., & Alhasanat, M. B. (2017). The History and Distribution of ṭāb: A Survey of Petra's Gaming Boards. *Journal of Near Eastern Studies*, *76*(1), 93–101. doi:10.1086/690502

Dorigo, M., & Birattari, M. (2011). Ant colony optimization. In Encyclopedia of machine learning (pp. 36-39).

Glenn, J. (2006). *An optimal strategy for Yahtzee. Loyola College*. Maryland: Loyola College in Maryland.

Hassanat, A. B., & Alkafaween, E. (2017). On enhancing genetic algorithms using new crossovers. *International Journal of Computer Applications in Technology*, *55*(3), 202–212. doi:10.1504/IJCAT.2017.084774

Hassanat, A. B., Alkafaween, E., Al-Nawaiseh, N. A., Abbadi, M. A., Alkasassbeh, M., & Alhasanat, M. B. (2016). Enhancing Genetic Algorithms using Multi Mutations: Experimental Results on the Travelling Salesman Problem. *International Journal of Computer Science and Information Security*, *14*(7), 785–802.

Hassanat, A., Prasath, V., Abbadi, M., Abu-Qdari, S., & Faris, H. (2018). An Improved Genetic Algorithm with a New Initialization Mechanism Based on Regression Techniques. *Information*, *9*(7), 167–197. doi:10.3390/info9070167

Holland, J. H. (1992). *Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence*. MIT press.

Jagiello, J., Eronen, M., Tay, N., Hart, D., Warne, L., & Hasan, H. (2006). Simulation Framework as a Multi-User Environment for a Go*Team game. In *37th Annual Conference of the International Simulation and Gaming Association*, St Petersburg, Russia (pp. 3-7).

Kennedy, J., & Eberhart, R. (1995). Particle Swarm Optimization. In *IEEE International Conference on Neural Networks IV* (pp. 1942–1948). IEEE. doi:10.1109/ICNN.1995.488968

Murray, H. J. (1952). *A history of board-games other than chess*. Clarendon press.

Oh, D. Y. (2017). Experiments to Parameters and Base Classifiers in the Fitness Function for GA-Ensemble. *International Journal of Statistics in Medical and Biological Research, 1*(1), 9-18.

Revello, T. E., & McCartney, R. (2002). *Generating war game strategies using a genetic algorithm. In Evolutionary Computation, CEC02. 2* (pp. 1086–1091). IEEE.

Simon, H. A., & Schaeffer, J. (1992). The game of chess. Handbook of game theory with economic applications, 1(1), 1-17.

Sun, C. T., Liao, Y. H., Lu, J. Y., & Zheng, F. M. (1994). Genetic algorithm learning in game playing with multiple coaches. In *IEEE World Congress on Computational Intelligence* (pp. 239-243). IEEE.

Tesauro, G. (2002). Programming backgammon using self-teaching neural nets. *Artificial Intelligence*, *134*(1), 181–199. doi:10.1016/S0004-3702(01)00110-2

Yang, X. S., & Deb, S. (2009). *Cuckoo search via Lévy flights. Nature & Biologically Inspired Computing* (pp. 210–214). IEEE.

Yasumura, Y., Oguchi, K., & Nitta, K. (2001). Negotiation strategy of agents in the Monopoly game. In *IEEE International Symposium on Computational Intelligence in Robotics and Automation* (pp. 277-281). IEEE. doi:10.1109/CIRA.2001.1013210

Ahmad B. A Hassanat was born and grew up in Jordan, and received his Ph.D. in Computer Science from the University of Buckingham at Buckingham, UK in 2010, and B.S. and M.S. degrees in Computer Science from Mutah University/Jordan and Al al-Bayt University/Jordan in 1995 and 2004, respectively. He has been a faculty member of Information Technology department at Mutah University since 2010. His main interests include computer vision; Machine learning, Big data analysis, Evolutionary algorithms and AI.

Hossam Faris is an Associate professor at Business Information Technology department/King Abdullah II School for Information Technology/ The University of Jordan (Jordan). Hossam Faris received his BA, M.Sc. degrees (with excellent rates) in Computer Science from Yarmouk University and Al-Balqa` Applied University in 2004 and 2008 respectively in Jordan. Since then, he has been awarded a full-time competition-based PhD scholarship from the Italian Ministry of Education and Research to peruse his PhD degrees in e-Business at University of Salento, Italy, where he obtained his PhD degree in 2011. In 2016, he worked as a Postdoctoral researcher with GeNeura team at the Information and Communication Technologies Research Center (CITIC), University of Granada (Spain). His research interests include: Applied Computational Intelligence, Evolutionary Computation, Knowledge Systems, Data mining, Semantic Web and Ontologies.

Mahmoud B. Alhasanat is an Associate professor with Civil Engineering department, Al-Hussein Bin Talal University, Maan.

Alex de Voogt's research interests are diverse and include the fields of psychology, history and anthropology. His research on games has focused on mancala board games—a group characterized by rows of holes and a proportionate number of identical playing counters—as well as on the expertise of master players. After several years as a curator of African Ethnology at the American Museum of Natural History, he is currently an associate professor of business at Drew University.

Surya Prasath is an assistant professor in the Division of Biomedical Informatics at the Cincinnati Children's Hospital Medical Center, USA. He received his PhD in Mathematics from the Indian Institute of Technology Madras, India in 2009 (defended in March 2010). He has been a postdoctoral fellow at the Department of Mathematics, University of Coimbra, Portugal, for two years. From 2012 to 2017 he was with the Computational Imaging and VisAnalysis (CIVA) Lab at the University of Missouri, USA and worked on various mathematical image processing and computer vision problems. He had summer fellowships/visits at Kitware Inc. NY, USA, The Fields Institute, Canada, and IPAM, University of California Los Angeles (UCLA), USA. His main research interests include nonlinear PDEs, regularization methods, inverse & ill-posed problems, variational, PDE based image processing, and computer vision with applications in remote sensing, biomedical imaging domains.